PROYECTO – Entrega 5

DIAGRAMA ENTIDAD-RELACIÓN

- Son representaciones gráficas que muestran cómo está estructurada una base de datos, es decir, qué entidades (tablas) existen y cómo se relacionan entre sí mediante claves primarias y foráneas. Sirven para diseñar y comprender la base de datos antes de implementarla.

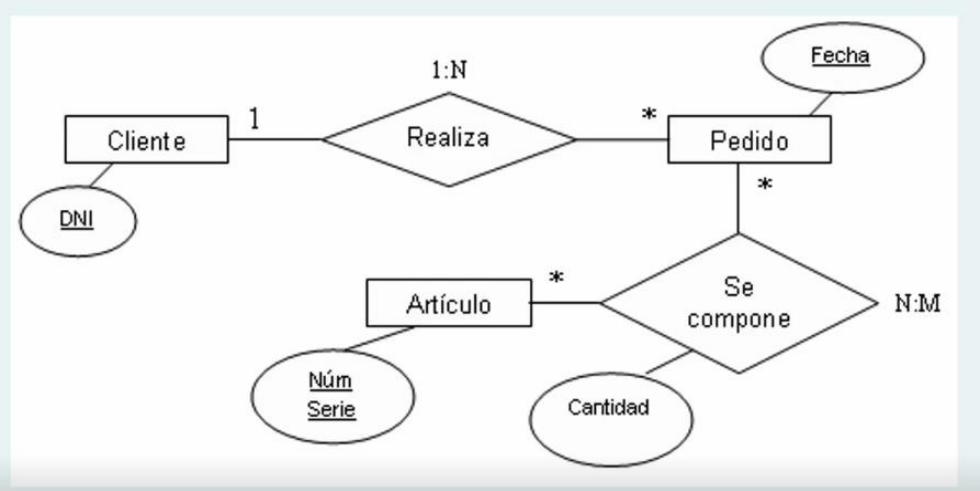


DIAGRAMA ENTIDAD-RELACIÓN

- **1:1 (uno a uno)**

Una entidad A se relaciona con una sola entidad B, y viceversa.
Ejemplo:
Cada persona tiene un solo DNI, y cada DNI pertenece a una sola persona.

- **1:N (uno a muchos)**

Una entidad A puede relacionarse con varias entidades B, pero cada B solo con una A.
Ejemplo:
Un cliente puede hacer muchos pedidos, pero cada pedido pertenece a un solo cliente.

- **N:1 (muchos a uno)**

Es la inversa de la anterior.
Ejemplo:
Muchos pedidos pertenecen a un cliente.

- **N:M (muchos a muchos)**

Varias entidades A se relacionan con varias entidades B.
Ejemplo:
Un estudiante puede estar en muchos cursos, y un curso puede tener muchos estudiantes.

DIAGRAMA ENTIDAD-RELACIÓN

Ejemplo:

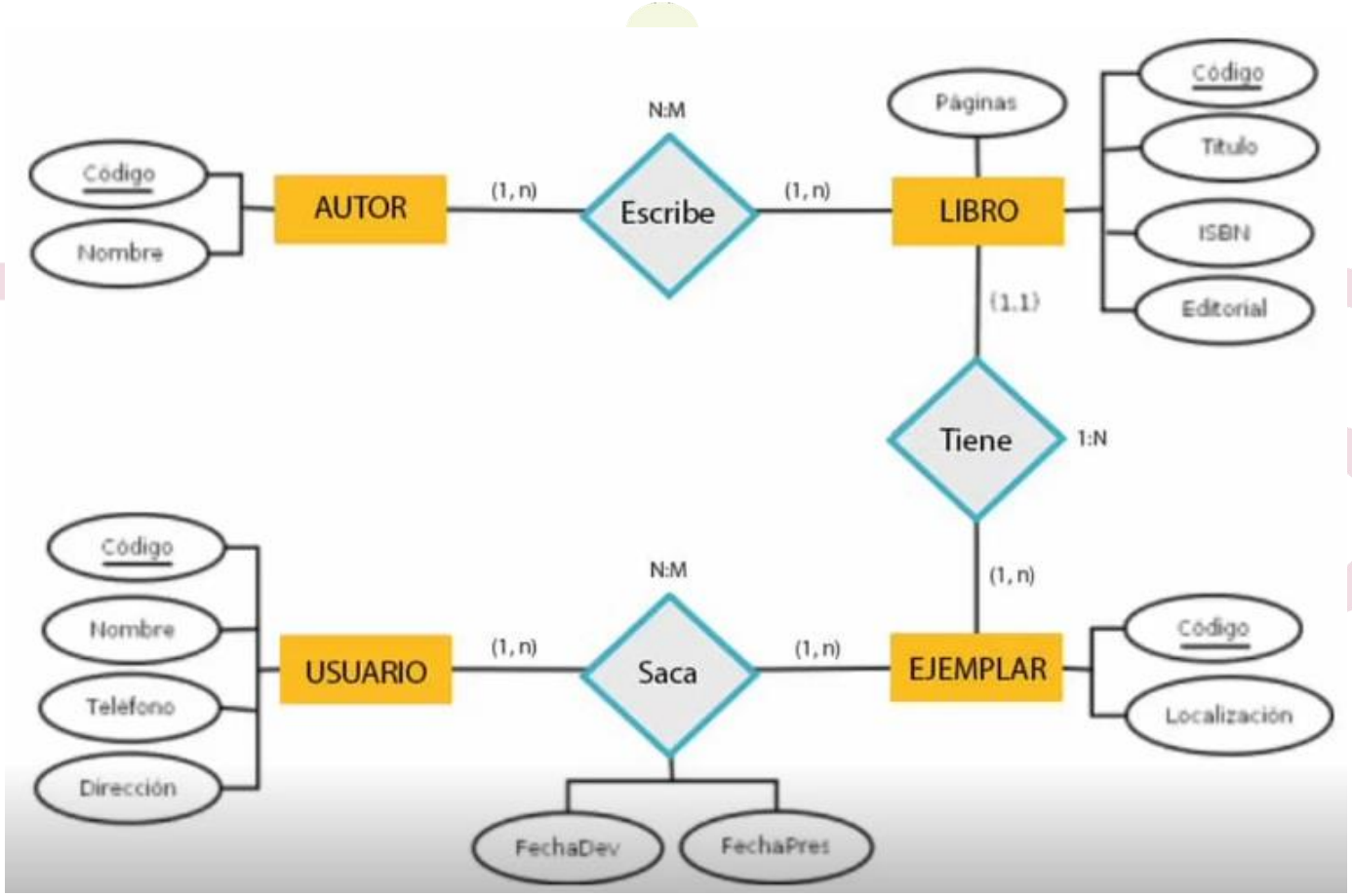


DIAGRAMA DE TABLAS

- Este tipo de diagrama muestra cómo están conectadas las tablas (entidades) dentro de una base de datos relacional.

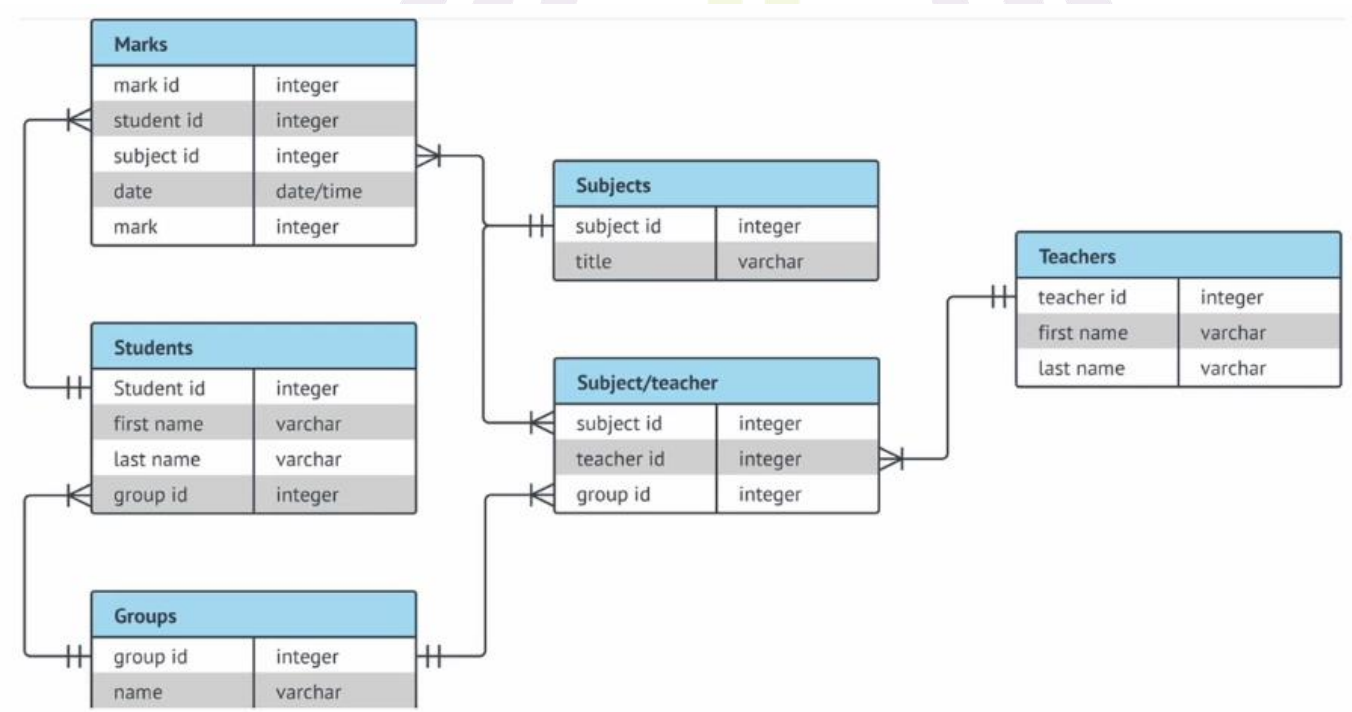


DIAGRAMA DE TABLAS

- Y dentro de la representación anterior podemos definir las relaciones entre distintas tablas (cardinalidad)

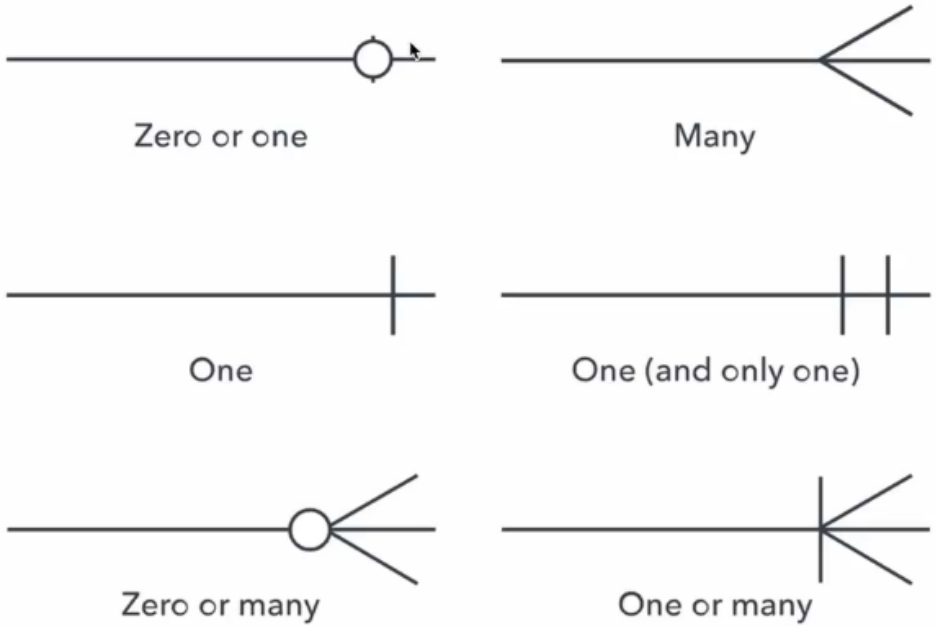


DIAGRAMA DE TABLAS

1. Cero o uno (Zero or one): Indica una relación **opcional**. Un registro en la tabla A puede no estar relacionado con ninguno en la tabla B, o como máximo, estar relacionado con **uno**.
   - **Ejemplo:** Un Empleado y un Coche de empresa (algunos empleados tienen uno, otros ninguno).

2. Muchos (Many) Es el símbolo genérico para representar una cantidad **indefinida** de registros. Aunque rara vez se usa solo (suele ir acompañado de un círculo o una línea), indica que no hay un límite superior estricto de conexiones.

3. Uno (One) Representa una relación con una **única instancia**. Es una forma simplificada de decir que existe una conexión.

4. Uno (y solo uno) (One and only one) Indica una relación **obligatoria y única**. El registro de la tabla A **debe** estar relacionado exactamente con un registro de la tabla B. Ni cero, ni dos; exactamente uno.
   - **Ejemplo:** Una Persona y su Acta de Nacimiento.

5. Cero o muchos (Zero or many) Es una relación **opcional**. Un registro en la tabla A puede no tener ninguna relación en la tabla B, o puede tener muchísimas. Es muy común en bases de datos flexibles.
   - **Ejemplo:** Un Cliente y sus Pedidos (un cliente nuevo puede no tener pedidos aún).

6. Uno o muchos (One or many) Es una relación **obligatoria**. Indica que debe existir **al menos uno**, pero no hay límite máximo. No se permite que el registro exista sin al menos una conexión.
   - **Ejemplo:** Una Factura y sus Líneas de Detalle (una factura no puede existir si no tiene al menos un producto listado).

DIAGRAMA DE TABLAS

- A continuación se muestra un ejemplo completo de diagrama entidad-relacion:

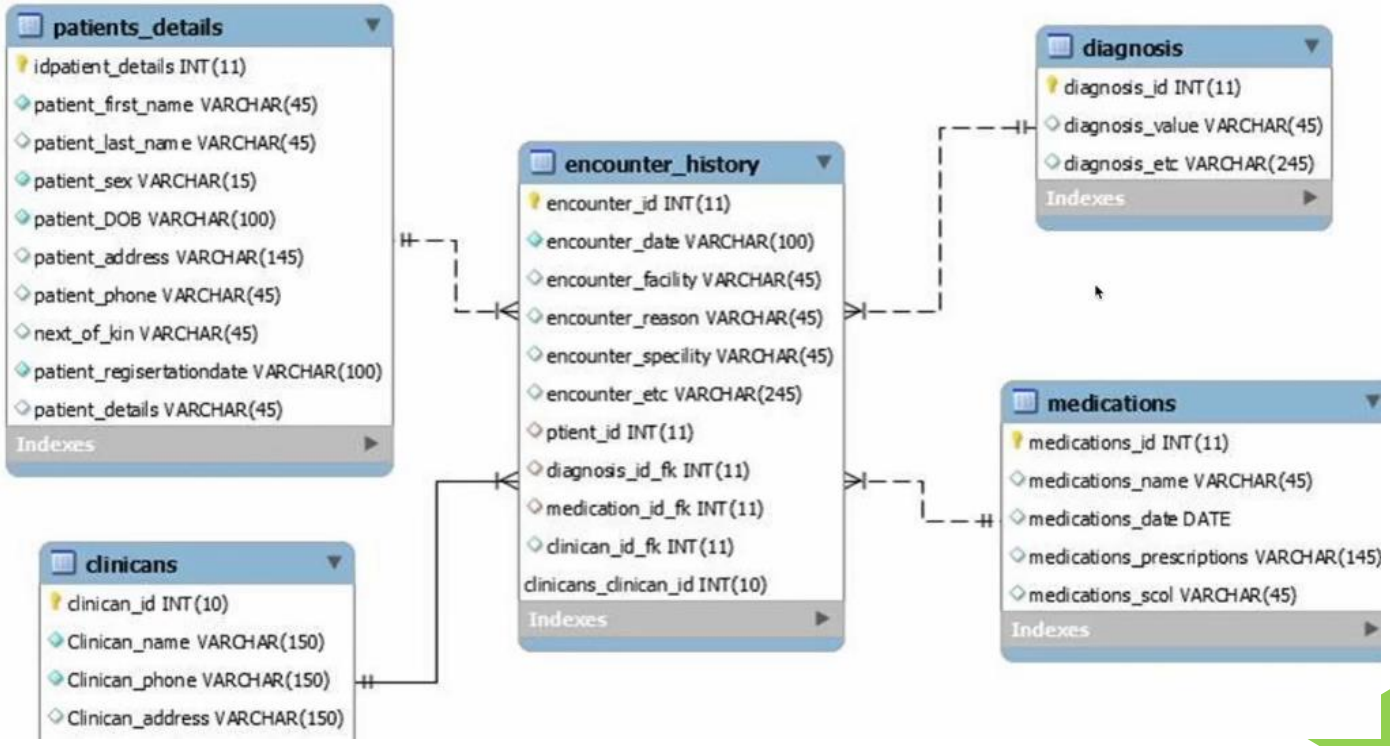


DIAGRAMA DE TABLAS – HERRAMIENTA DRAW.IO

- Para crear nuestros diagramas, podemos utilizar draw.io:
- https://app.diagrams.net/

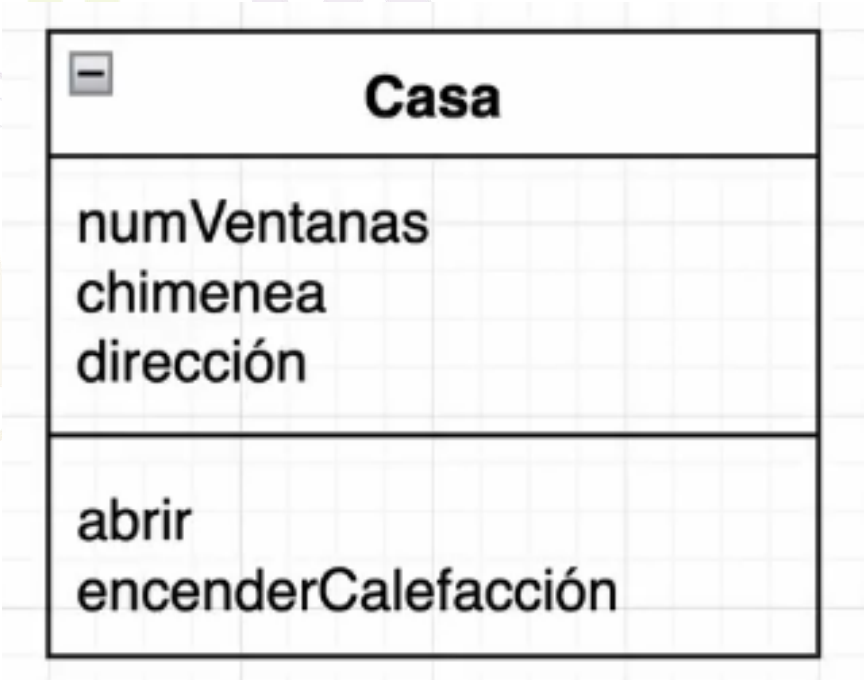


DIAGRAMA DE TABLAS – HERRAMIENTA DRAW.IO

- **La clase Animal**, define el modelo general de un objeto que representa a un animal.
- Debajo se encuentran los **atributos**, que son las características que cada objeto de esta clase tendrá. En este caso hay tres: `name`, `id` y `age`. Junto a cada uno hay un símbolo que indica su nivel de acceso.
- El signo # significa que el atributo es **protegido**, es decir, puede ser usado por la propia clase y por sus clases hijas.
- El signo - indica que el atributo es **privado**, por lo que solo puede usarse dentro de la clase.

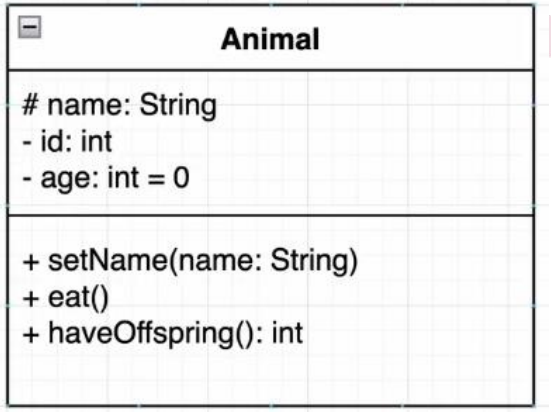
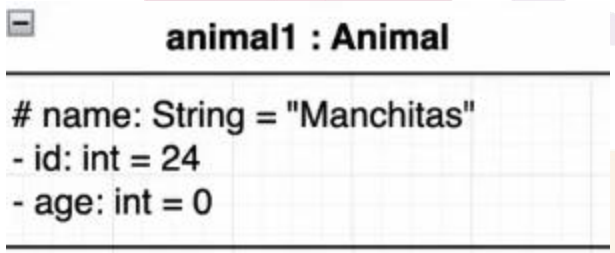


DIAGRAMA DE TABLAS – HERRAMIENTA DRAW.IO

- Desde nuestro lenguaje de programación, podría verse de la siguiente forma (si se usa java)




```
public class Imagen {
    // Atributos (campos de clase)
    private String name = "Manchitas";
    private int id = 24;
    private int age = 0;

    // Constructor
    public Imagen() {}

    public Imagen(String name, int id, int age) {
        this.name = name;
        this.id = id;
        this.age = age;
    }

    // Getters y Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```